

glftmm_lot 策略说明

基于当前仓库代码与配置

2026-06-16

目录

1 阅读入口	3
2 当前配置下的策略族	3
3 数据流和时间顺序	4
3.1 采样盘口	4
3.2 交易强度	4
3.3 波动率	5
3.4 Instructor	5
3.5 事件合并	5
4 单 lot 策略的报价公式	5
5 库存上限和普通开仓	6
6 挂单延迟、订单簿和撮合	7
6.1 mode=0 和 mode=1	7
6.2 成交方向	7
7 profit grid 止盈状态机	7
7.1 网格价格	8
7.2 网格数量分布	8
7.3 超过上界后的释放	9
7.4 重复网格保护	9
8 水下补仓数量曲线	9
9 止损和 reach-and-release	9
10 最大持仓时间	10
11 multi-lot 分层逻辑	10

11.1 初始状态	10
11.2 什么时候启用下一层 lot	11
11.3 谁可以开仓，谁只能平仓	11
11.4 trade 成交优先级	11
11.5 内层平空后的 index 轮换	12
11.6 聚合输出	12
12 仓位和 PnL	12
13 每日运行、恢复和提前停止	13
14 输出字段怎么读	13
15 当前配置下真正生效的机制	14
16 几个容易误读的边界	14
17 读结果时建议看的顺序	15

1 阅读入口

这份文档描述当前仓库里的 SimpleMakerStrategy，以及它在 config/config.json 下形成的策略族。主要代码入口是：

- run_strategy.py: 读取配置、展开参数组合、恢复每日状态、写出 parquet 和 summary。
- sim/strategy.py: 核心做市策略、profit grid、止损、超时、multi-lot 状态机。
- sim/loader.py: 把 sampler 缓存和原始成交流合成时间序列事件。
- core/manager.py、core/orderbook.py、core/position.py: 本地订单簿、撮合、仓位和 PnL。
- sampler/*.py: 采样盘口、交易强度、波动率、instructor 信号。

一句话概括：这是一个事件驱动的被动做市回测。策略在固定频率的盘口采样点更新双边报价，用原始成交流模拟自己的 maker/taker 成交；持仓盈利时改为 close-only 的分层止盈网格。multi-lot 版本把仓位组织成从内到外的 lot 栈，内层只允许平仓，只有最外层可开仓 lot 才按原单 lot 规则自由开平仓。

2 当前配置下的策略族

当前 config/config.json 的交易对象是 BTCUSDT，日期为 2025-01-02 到 2026-06-01。策略读取原始数据根目录 input_path 和 input_backup_path，读取 sampler 缓存根目录 output_path=/data/users/kang/backtest/glftmm_var/cached，策略结果写到 /data/users/kang/backtest/glftmm_lot/result。

本文描述策略运行时以 config/config.json 为准。sampler/config.json 当前自己的 output_path 是 /data/users/kang/backtest/glftmm_lot/cached；如果两份配置不同，策略实际读取的是 config/config.json 里的缓存路径。

当前会展开成 16 个策略任务：

- mode 有 2 个值：1 和 0。
- profit_grid 有 4 组。
- max_position_usdt 和 stoploss 成对，有 2 组 lot 结构。
- 其他参数当前都是单值。

配置项	当前含义
mode=[1,0]	撮合难度。0 是 inclusive, 1 是 strict。mode=1 更难成交。
latency=2	ticker 事件生成的 maker 报价在 2 毫秒后才进入本地 maker book。
price_precision=1	价格 tick 为 0.1。
qty_precision=3	数量 step 为 0.001。
taker_fee=0.00025	taker 成交扣 2.5 bps。
maker_fee=-0.00003	maker 成交是负手续费，即返佣 0.3 bps。
freq=1000	每 1000 ms 读取一次采样盘口和特征。

配置项	当前含义
name_intensity	/ 当前为 $k / 300$ ，用 300 个采样点的破价强度均值作为基础
lookback_intensity	报价距离。
name_volatility	/ 当前为 $\sigma / 300$ ，读取 σ 波动率，但当前
lookback_volatility	adj_spread_volatility=0，不改变报价。
name_instructor	/ 当前为 bbo_imbalance / 0，读取当前采样盘口深度不平衡，
lookback_instructor	但当前 adj_spread_instructor=0，不改变报价。
open_passive_only=true	如果将来启用 instructor，只允许它把普通开仓报价变得更
	被动，不允许追价开仓。当前因 instructor 缩放为 0，实际
	无影响。
order_amt=1000	每次基础单边下单名义金额约 1000 USDT，再按数量精度
	和最小下单约束截断。
min_order_notional=100	开仓部分必须满足最小名义金额。
inventory_skew=[0,1]	最大 skew tick 为 0，所以当前不开库存偏移。
open_curve_underwater	当前为 [0,1,0]，order 为 0，所以当前不开水下补仓数量衰
	减。
new_open_lot_crit=0.01	下一层 lot 只有在上一层满仓且价格相对上一层成本价逆向
	偏离 1% 后才允许开仓。
max_holding_time=-1	不启用持仓超时强平。
profit_grid	四组止盈网格，分别测试不同盈利带和数量分布。

当前 lot 结构如下：

- 五层 lot：每层 max_position_usdt=100000，每层 stoploss=30000，总仓位上限 500000 USDT。

3 数据流和时间顺序

策略运行前需要 sampler 缓存。当前策略本身不负责生成 sampler 缓存；它只读取已经存在的 parquet。如果缺少某一天的采样盘口、强度、波动率或 instructor 文件，策略会直接报错。

3.1 采样盘口

sampler/resample.py 把原始 BOOKTICKER 转成固定网格。每个采样时间戳使用该时刻及之前最后一条已知盘口：

$$\text{sampler_ticker}(t) = \text{last bookticker update at or before } t.$$

它不会使用 $[t, t + \text{freq})$ 内未来才出现的盘口。输出列包括：timestamp、best_bid_price、best_ask_price、best_bid_qty、best_ask_qty。

3.2 交易强度

当前策略使用 name_intensity=k。它的基础量是每个桶内成交价突破当时 BBO 的最大距离：

$$\text{break_dist} = \max(\max(\text{trade_price} - \text{best_ask}, 0), \max(\text{best_bid} - \text{trade_price}, 0)).$$

代码先对 `bucket_break_dist_max` 做 `shift(1)`，所以采样点 t 使用的是上一桶 $[t - \text{freq}, t)$ 已经发生的破价信息。指标 k 是非零破价距离的滚动均值；`k_decay` 是指数衰减；`k_median` 是非零破价距离滚动中位数。

3.3 波动率

`sampler/volatility.py` 先构造每个采样桶的 `mid`、`open`、`high`、`low`、`close`，然后支持多种波动率：`sigma`、`atr`、`rv`、`bv`、`parkinson`、`gk`。当前策略读取 `sigma`，但由于 `adj_spread_volatility=0`，它不会参与报价距离。

3.4 Instructor

支持两类 `instructor`：

- `trade_imbalance`：按成交方向聚合上一窗口的买卖量，先 `shift(1)`，再计算

$$\frac{\text{bid_qty} - \text{ask_qty}}{\text{bid_qty} + \text{ask_qty} + \epsilon}.$$

这个信号严格不使用同桶成交。

- `bbo_imbalance`：当前采样盘口深度不平衡：

$$\frac{\text{best_bid_qty} - \text{best_ask_qty}}{\text{best_bid_qty} + \text{best_ask_qty} + \epsilon}.$$

它只支持 `lookback=0`。

当前配置读取 `bbo_imbalance`，但 `adj_spread_instructor=0`，所以它不改变实际报价。

3.5 事件合并

`BinanceEventLoader` 读取三类数据：采样 `ticker/alpha` 行、原始 `trade` 行、可选的强度/波动率/`instructor` 列。它输出两种事件：

- `trade`：timestamp, is_buyer_maker, price, volume。
- `ticker`：timestamp, best_bid, best_ask, instructor, intensity, volatility。

如果 `trade` 和 `ticker` 时间戳相同，`trade` 先输出。策略处理每个事件前会先激活已经到期的 pending maker quote。因此，若一张挂单的 `active_ts` 正好等于某笔 `trade` 的时间戳，它可以先进入本地订单簿，再参与这笔 `trade` 的撮合；该时间戳的 `ticker` 更新会在 `trade` 之后处理。

4 单 lot 策略的报价公式

每个 `ticker` 事件先检查 BBO 是否有效：`bid`、`ask` 必须有限且为正，并且 `bid < ask`。中间价为：

$$\text{mid}_t = \frac{\text{best_bid}_t + \text{best_ask}_t}{2}.$$

基础报价距离为：

$$d_t = intensity_t \cdot adj_spread_intensity + volatility_t \cdot adj_spread_volatility.$$

当前配置下, $adj_spread_intensity=1$ 且 $adj_spread_volatility=0$, 所以:

$$d_t = intensity_t.$$

Instructor 的方向位移为:

$$shift_t = instructor_t \cdot adj_spread_instructor \cdot mid_t.$$

如果 $open_passive_only=true$, 普通开仓报价会做 clipping:

$$shift_t^{ask} = \max(0, shift_t), \quad shift_t^{bid} = \min(0, shift_t).$$

这意味着 instructor 只能让 ask 往上、bid 往下, 即让开仓更被动; 不会让 ask 往下或 bid 往上追价。当前配置 $adj_spread_instructor=0$, 所以 $shift_t = 0$ 。

库存偏移为:

$$skew_t = \begin{cases} -[|u_t|^p \cdot max_skew_ticks] \cdot tick, & u_t > 0, \\ [|u_t|^p \cdot max_skew_ticks] \cdot tick, & u_t < 0, \\ 0, & u_t = 0, \end{cases}$$

其中 $u_t = cost_notional_usdt / max_position_usdt$ 。多头库存会把双边报价往下移, 鼓励卖出、抑制继续买入; 空头库存会把双边报价往上移。当前 $max_skew_ticks=0$, 所以不开库存偏移。最终普通报价为:

$$ask_price = ceil_tick(best_ask + d_t + shift_t^{ask} + skew_t),$$

$$bid_price = floor_tick(best_bid - d_t + shift_t^{bid} + skew_t).$$

基础单边数量为:

$$q = floor_step(order_amt / mid_t).$$

如果 q 小于 min_order_qty 或 $min_order_notional / mid_t$, 则该侧数量变成 0。当前 $order_amt=1000$, $min_order_notional=100$, 所以一般情况下基础订单不会被最小名义金额挡住。

5 库存上限和普通开仓

单 lot 的仓位上限使用 $cost_notional_usdt$ 判定, 不使用 $mark_notional$:

$$cost_notional_usdt = qty \cdot cost.$$

规则是:

- 如果 $cost_notional_usdt \geq max_position_usdt$, 停止继续挂 bid 开多。
- 如果 $cost_notional_usdt \leq -max_position_usdt$, 停止继续挂 ask 开空。
- 如果 $max_position_usdt \leq 0$, 双边数量都为 0。

注意这里是方向性 cost notional。多头达到上限时不再买, 但仍可卖; 空头达到上限时不再卖, 但仍可买。

6 挂单延迟、订单簿和撮合

Ticker 事件生成的 maker levels 不会立即永久替换订单簿, 而是先进入 pending 队列:

$$active_ts = timestamp + latency.$$

每个新事件开始时, 策略会把所有已到期的 pending quote 弹出, 只保留最后一条到期报价, 清空旧 maker book, 然后放入这条报价。这样做的效果是: 如果市场事件很稀疏, 中间过期的旧报价不会逐条执行, 只有最新到期报价生效。

6.1 mode=0 和 mode=1

core/orderbook.py 里 mode 决定成交阈值:

- ask 挂单: mode=0 时 $ask_price \leq trade_price$ 成交; mode=1 时 $ask_price < trade_price$ 才成交。
- bid 挂单: mode=0 时 $bid_price \geq trade_price$ 成交; mode=1 时 $bid_price > trade_price$ 才成交。

所以 mode=1 比 mode=0 更难成交, 尤其是在 trade price 正好等于挂单价时, mode=1 不会填。

6.2 成交方向

原始 trade 的 is_buyer_maker=true 表示买方是 maker, 主动方是卖方, 也就是市场卖单打到了被动 bid。策略注释中直接写了这一点。对应关系是:

- is_buyer_maker=true: 公共卖流打 bid, 本策略的 bid maker 可能成交, 仓位增加。
- is_buyer_maker=false: 公共买流打 ask, 本策略的 ask maker 可能成交, 仓位减少或开空。

每笔 trade 先尝试撮合 maker book。若成交量还有剩余, 再撮合 taker book。maker 成交用 maker_fee, taker 成交用 taker_fee。

7 profit grid 止盈状态机

当前策略开启 profit_grid, 所以持仓后的行为不是继续双边普通报价, 而是根据持仓盈亏切换:

- 空仓：正常双边开仓。
- 多头且 $mid \leq cost$ ：仍处于水下或刚回本，只挂 bid 继续补多，不挂 ask 止盈。
- 多头且 $mid > cost$ ：只挂 ask close-only 止盈网格，不再挂 bid 开仓。
- 空头且 $mid \geq cost$ ：仍处于水下或刚回本，只挂 ask 继续补空，不挂 bid 止盈。
- 空头且 $mid < cost$ ：只挂 bid close-only 止盈网格，不再挂 ask 开仓。

也就是说，profit grid 把单 lot 的行为分成两个区域：

- 水下区域：允许沿持仓方向继续补仓，但受仓位上限和 underwater 曲线影响。
- 盈利区域：只做 close-only 分层止盈。

7.1 网格价格

`profit_grid=[lower_bps, upper_bps, num_grid, qty_kind, qty_param]`。多头止盈价格在：

$$cost \cdot (1 + lower/10000) \text{ 到 } cost \cdot (1 + upper/10000)$$

之间均匀取 `num_grid` 档，并向上取 tick。空头止盈价格在：

$$cost \cdot (1 - lower/10000) \text{ 到 } cost \cdot (1 - upper/10000)$$

之间均匀取档，并向下取 tick。

如果多头某个止盈价低于当前 `best_ask`，会被抬到 `best_ask`；如果空头某个止盈价高于当前 `best_bid`，会被压到 `best_bid`。这避免 close-only 网格挂在明显不合理的位置。

7.2 网格数量分布

可选数量分布有三类：

- `equal`：每档同权。
- `power`：权重为 $(num_grid - index)^{param}$ ，越靠近第一档权重越大。
- `exponential`：权重为 $\exp(-param \cdot index)$ ，`param` 越大，越集中在第一档。

代码先按权重分配总平仓数量，再按 `step` 向下取整。若某档数量小于最小合法下单量，则后续档不再分配。最后剩余数量会加到第一档。因此第一档通常承担最多数量，特别是在 `exponential` 分布下。

当前配置的四组网格是：

- `[5,105,10,"exponential",0.2]`：5 到 105 bps，10 档，数量缓慢向第一档集中。
- `[5,105,10,"exponential",0.5]`：同样盈利带，数量更集中在第一档。
- `[2,52,10,"exponential",0.2]`：更近的 2 到 52 bps，10 档。
- `[2,22,5,"exponential"]`：2 到 22 bps，5 档，`exponential` 默认参数为 1。

7.3 超过上界后的释放

如果 mid 已经超过 profit grid 上界, 策略不再维持整条网格, 而是生成一档更容易成交的 close-only 单:

- 多头: 价格为 $\min(\text{ceil}(\text{best_bid} + \text{tick}), \text{best_ask})$ 。
- 空头: 价格为 $\max(\text{floor}(\text{best_ask} - \text{tick}), \text{best_bid})$ 。

这不是市价单, 但比原网格更接近盘口, 用于在价格已经穿过目标盈利区后尽快释放持仓。

7.4 重复网格保护

close-only 网格有一个 profit_grid_key, 由 ask 价格序列、bid 价格序列和持仓成本组成。如果新 ticker 计算出的 close-only 网格和上一条完全相同, 策略不会清空并重挂 maker book, 只记录状态。这减少了重复改单。

8 水下补仓数量曲线

当 profit grid 开启且持仓在水下时, 补仓数量会通过 open_curve_underwater 乘一个系数:

$$util = \min(1, \max(0, \text{gross_cost_notional_usdt} / \text{max_position_usdt})),$$

$$\text{multiplier} = \text{min_scale} + (\text{max_scale} - \text{min_scale}) \cdot (1 - util)^{\text{order}}.$$

如果 order ≤ 0, 代码直接返回 1。当前配置为 [0, 1, 0], 所以当前不会随仓位利用率降低补仓数量。

如果将来把 order 调成正数, 仓位越接近该 lot 的上限, 补仓数量越小; 当 util 接近 1 时, multiplier 接近 min_scale。

9 止损和 reach-and-release

单 lot 的止损按未实现浮亏触发:

$$\text{floating_unrealized} = \text{qty} \cdot (\text{mid} - \text{cost}),$$

$$\text{floating_loss} = \max(0, -\text{floating_unrealized}).$$

如果 floating_loss ≥ stoploss, 策略进入 reach_and_release_active。进入该状态后:

- 清空 pending maker quote。
- 清空 maker book。
- 清空旧 taker book。
- 按当前仓位方向放一张 close-only taker 单。

多头止损时，策略在 ask taker book 放卖出单，价格用最新 `best_bid`；空头止损时，策略在 bid taker book 放买入单，价格用最新 `best_ask`。它不是在 ticker 事件当场直接改仓，而是等后续 trade 事件来撮合 taker book。成交后用 `taker_fee` 计费。

当仓位完全归零后，`reach_and_release_active` 清除，超时跟踪也清除。

10 最大持仓时间

`max_holding_time` 控制达到单 lot 仓位上限后的超时释放：

- `max_holding_time<0`：关闭该逻辑。当前配置为 -1。
- `max_holding_time=0`：只要已经到达上限并记录了开始时间，下一个 ticker 就触发释放。
- `max_holding_time>0`：达到上限后，如果当前时间戳减开始时间大于该值，则触发释放。

只要仓位被减小，超时开始时间会重置。这里的“达到上限”同样用 `gross_cost_notional_usdt >= max_position_usdt` 判定。

11 multi-lot 分层逻辑

`glftmm_lot` 和普通单 lot 策略的主要区别在这里。配置中的每个 `max_position_usdt` 元素都会生成一个 `_SingleLotMakerStrategy`，并配对一个同位置的 `stoploss`。这些子策略共享同一条市场事件流，但仓位、成本、订单簿、止盈网格、止损状态分别保存。

当前 multi-lot 不是“多个 lot 各自独立交易”。它被组织成一个从内到外的逻辑栈：

- index 小的是内层 lot，index 大的是外层 lot。
- `active_lot_count` 表示当前开放的前缀长度；`active_lot_indices` 是 `0..active_lot_count-1`。
- `frozen_lot_indices` 是尚未开放的外层备用 lot。

11.1 初始状态

多 lot 初始化时：

- `active_lot_count=1`。
- `active_lot_indices=[0]`，只有最内层 lot 开始工作。
- 其他 lot 在 `frozen_lot_indices` 中，作为外层备用 lot。

每个事件开头，策略会先激活各 lot 已经到期的 pending maker quote。但 trade 事件不会再让所有 lot 各自独立吃同一笔成交；它只在 active 前缀内，从内到外依次撮合，并把上一层已经成交的数量从这笔 trade 中扣掉。

11.2 什么时候启用下一层 lot

每个 ticker 事件前，策略先同步 lot 栈。如果当前 active 前缀的最外层 lot 已经有仓位，并且满足：

- 该 lot 已经达到自己的仓位上限；
- 判断标准是 $\text{gross_cost_notional_usdt} \geq \text{max_position_usdt}$ ；
- 当前价格相对该 lot 成本价达到 new_open_lot_crit 的逆向偏离；
- 多头满仓时要求 $\text{mid} \leq \text{cost} * (1 - \text{new_open_lot_crit})$ ，空头满仓时要求 $\text{mid} \geq \text{cost} * (1 + \text{new_open_lot_crit})$ ；
- 这个价格门槛只控制下一层是否能开仓，不改变当前 lot 的 stoploss 和 close-only 流程；

则 active_lot_count 加 1，下一层外侧备用 lot 被开放。这个开放发生在 ticker 事件上；如果某一笔 trade 刚好让当前外层 lot 达到满仓，下一层会在后续 ticker 同步时检查满仓和价格偏离是否同时满足。

因此，多 lot 不是同时开所有层，而是只有上一层达到任意方向满仓，并且行情继续朝该层不利方向移动足够远后才启用下一层。这个机制把加仓风险拆成一层一层向外展开的仓位单元，并降低单边行情里快速打满所有 lot 的速度。

11.3 谁可以开仓，谁只能平仓

ticker 事件上，multi-lot 会先找当前可开仓 lot：

- 如果 active 前缀里没有任何仓位，则第 0 层可以按原单 lot 规则双边开仓。
- 如果已经有非空 lot，通常只有最外层非空 lot 可以按原单 lot 规则自由开平仓。
- 如果最外层非空 lot 已经满仓、价格偏离条件仍然满足，并且外侧存在一个刚开放但仍为空的备用 lot，则这个空备用 lot 是当前可开仓 lot。
- 其他更内层的非空 lot 不允许开仓，只能保留现有规则下的减仓侧报价。例如多头只保留 ask 平仓侧，空头只保留 bid 平仓侧。

这意味着内层 lot 在水下时不会继续补仓；它要么等待现有规则产生可平仓报价，要么进入 stoploss / reach-and-release 的 close-only 流程。外层 lot 承担新的开仓和补仓动作。

11.4 trade 成交优先级

一笔 trade 到来时，active 前缀按逻辑 index 从小到大处理，也就是从最内层开始：

$$\text{remaining_qty}_0 = \text{trade_qty}.$$

第 i 层只最多撮合 remaining_qty_i 。如果该层成交了 filled_i ，下一层只看到：

$$\text{remaining_qty}_{i+1} = \text{remaining_qty}_i - \text{filled}_i.$$

因此，同一笔市场成交会优先满足最内层的平仓单，再轮到次内层，以此类推。只有内层没有成交或成交后还有剩余数量，外层开仓单才可能吃到这笔 trade。

11.5 内层平空后的 index 轮换

同步 lot 栈时, 如果 active 前缀里某个内层 lot 已经空仓, 而它外侧仍有非空 lot, 则这个空 lot 会被移动到 active 前缀最外层的位置。它外侧的非空 lot 逻辑 index 依次前移。

例如 active 前缀原来是:

$$[lot_0 = \text{空}, lot_1 = \text{非空}, lot_2 = \text{非空}]$$

同步后变成:

$$[lot_1 = \text{非空}, lot_2 = \text{非空}, lot_0 = \text{空备用}]$$

这样可以保证非空仓位总是从内到外连续排列, 内层平仓优先级也随之更新。所有 frozen 且空仓的 lot 会清空 pending maker quote、maker book、taker book 和重复网格 key。

11.6 聚合输出

每日输出不是逐 lot 分开写 parquet, 而是把所有 lot 聚合成一条策略曲线:

- position: 各 lot 数量求和。
- mark_notional_usdt: 各 lot mark notional 求和。
- cost_notional_usdt: 各 lot cost notional 求和。
- gross_cost_notional_usdt: 各 lot gross cost notional 求和。
- realized_pnl 和 unrealized_pnl: 各 lot 求和。
- traded_volume: 各 lot 成交额求和。

状态 JSON 里会保留 lots, 包含每一层 lot 的独立状态。这里的 lots 顺序是当前逻辑顺序, 而不一定是最初配置顺序。状态也会保存 active_lot_count、active_lot_indices、frozen_lot_indices、max_position_lots、stoploss_lots 和 new_open_lot_crit, 用于断点续跑和输出复盘。

12 仓位和 PnL

Position.execute 使用 signed quantity:

- 正数表示买入, 仓位增加。
- 负数表示卖出, 仓位减少或转为空。

同方向加仓会更新平均成本。反方向成交会先平已有仓位并产生 realized PnL; 如果反向成交超过原仓位, 剩余部分以成交价作为新方向的成本。手续费直接从累计 realized PnL 中扣除。由于当前 maker fee 为负数, maker 成交会增加 realized PnL。

未实现 PnL 为:

$$unrealized_pnl = mark_notional_usdt - cost_notional_usdt.$$

其中:

$$mark_notional_usdt = qty \cdot mid, \quad cost_notional_usdt = qty \cdot cost.$$

13 每日运行、恢复和提前停止

run_strategy.py 以“每日”为增量单位运行。每个参数组合的输出目录形如:

<output_dir>/<symbol>/<sim_dir>/strat__all

每一天会写两个文件:

- YYYY-MM-DD.parquet: 该日内每个 ticker 事件后的聚合曲线。
- _state/YYYY-MM-DD.json: 该日结束后的可恢复状态。

如果 overwrite=false, 已经同时存在 parquet 和 state 的日期会跳过; 策略会从第一个缺失日期继续跑, 并用上一日 state 恢复。

策略有三类提前停止规则, 都是在 state 中记录:

- stop_due_to_dayn_pnl: 运行天数大于 60 且 total PnL 小于 0。
- stop_due_to_low_annualized_return: 运行天数大于 60 且 realized PnL 年化收益率低于 0.2。
- stop_due_to_max_drawdown: 相对历史最高 total PnL 的回撤除以总 max_position_usdt 小于等于 -0.2。

这里的年化收益率为:

$$\frac{realized_pnl}{max_position_usdt} \cdot \frac{365}{day_n}.$$

这里的 max_position_usdt 对多 lot 来说是所有 lot 上限之和。

14 输出字段怎么读

每日 parquet 字段如下:

字段	含义
timestamp	ticker 事件时间戳, 毫秒。
price	当前 mid price。
position	聚合仓位数量, 正数为多, 负数为空。
mark_notional_usdt	仓位按当前 mid 标记的方向性名义金额。

字段	含义
<code>cost_notional_usdt</code>	仓位按平均成本计算的方向性名义金额。
<code>gross_cost_notional_usdt</code>	<code>abs(cost_notional_usdt)</code> 。
<code>realized_pnl</code>	已实现 PnL，包含手续费。
<code>unrealized_pnl</code>	未实现 PnL。
<code>traded_volume</code>	策略自身累计成交额，所有 lot 求和。

`strategy_summary.csv` 汇总每个参数组合的最终指标，包括 total PnL、realized/unrealized PnL、最终仓位、成交额、按成交额折算的 PnL bps 等。需要注意：如果某个组合提前停止，summary 反映的是停止日状态；如果历史 summary 和当前配置不匹配，应优先看输出目录下的 `_state`。

15 当前配置下真正生效的机制

当前配置里，实际改变交易行为的主要是：

- `mode=0/1`：影响成交难度。
- `latency=2`：影响挂单进入 book 的时刻。
- `intensity=k`, `lookback=300`, `adj_spread_intensity=1`：决定普通报价离 BBO 的距离。
- `order_amt=1000` 和最小下单约束：决定普通开仓数量。
- `max_position_usdt` 和 `stoploss` 的多 lot 向量：决定分层仓位与每层止损。
- `new_open_lot_crit=0.01`：决定下一层 lot 需要相对上一层成本价逆向偏离 1% 才能开仓。
- `profit_grid`：决定盈利区的 close-only 分层止盈。
- 每日状态恢复和三类提前停止规则：决定长区间批量回测是否继续。

当前配置里，被读取但不会改变报价的机制是：

- `bbo_imbalance_instructor`：因为 `adj_spread_instructor=0`。
- `sigma` 波动率：因为 `adj_spread_volatility=0`。
- `inventory_skew`：因为最大 tick 为 0。
- `open_curve_underwater`：因为 `order` 为 0。
- `max_holding_time`：因为值为 -1。

16 几个容易误读的边界

- **profit grid 不是止损**。它只在持仓盈利时挂 close-only 网格。亏损时只有当前可开仓 lot 仍可能补仓；内层非空 lot 只允许平仓。
- **止损不是立即改仓**。止损触发后放 taker close-only 单，仍需要后续 trade 事件撮合。

- **多 lot 不是简单提高单层 max position，也不是多个独立策略。**每一层有独立成本、独立订单簿、独立止损；但开仓权限、成交数量、下一层价格门槛和 index 轮换由外层 lot 栈统一调度。
- **仓位上限用成本名义金额。**它不随 mark price 波动直接变化，除非成交改变平均成本或数量。
- **mode=1 更保守。**等价格不成交，只有严格穿过才成交。
- **当前 instructor 不生效。**文档里解释了 instructor 公式，但当前参数缩放为 0。
- **latency 后只保留最后一条到期报价。**高频 ticker 下旧 pending quote 会被新 quote 覆盖，不是每条过期报价都进入 book。

17 读结果时建议看的顺序

阅读某个结果目录时，建议按下面顺序：

1. 先看路径里的 `sim_...` 参数 token，确认它是否属于当前配置族。
2. 看最后一个 `_state/*.json`，确认 `day_n`、`total_pnl`、三个 stop 标记、`max_pnl_ever`。
3. 如果是 multi-lot，展开 state 里的 `lots`，看当前逻辑顺序下每层 lot 的仓位、成本、止损状态、`active_lot_count` 和 `active/frozen` 列表。
4. 再看每日 `parquet` 曲线，重点看 `position`、`cost_notional_usdt`、`gross_cost_notional_usdt`、`realized_pnl`、`unrealized_pnl`。
5. 最后再用 `strategy_summary.csv` 排名。`summary` 适合批量比较，但排查机制时不如 state 直接。